MILO

# Backend Data & Database Architecture

Complete data model, table catalogue, relationships, authentication and local identity flows, persistence rules, security boundaries, migrations, retention, and implementation contracts.

**Specification:** v1.0 **Platform:** Windows-first desktop application **Architecture:** Tauri + Rust + React + SQLite

**Prepared:** 02 November 2026 **Status:** Engineering baseline / implementation-ready draft

**Foundational decision:** MILO is local-first and account-free for its core product. A Windows user profile provides the operating-system identity boundary; MILO creates a local installation/profile identity for data ownership. Online authentication is optional and only exists if a future sync or account feature is explicitly introduced.

# Document control

| Item | Decision |
|---|---|
| Product | MILO — floating desktop island / productivity and system deck |
| Primary target | Windows 10/11; Windows-first implementation |
| Runtime | Tauri v2 shell, Rust backend, React frontend |
| Primary persistence | SQLite database stored in the current Windows user's application data directory |
| Identity model | No MILO username/password required for local use; local installation profile + Windows user boundary |
| Secrets | Windows DPAPI user scope or Windows Credential Manager; never plaintext in SQLite |
| File staging | Metadata-only references to original files; optional MILO-managed temporary copies only when explicitly requested |
| Cloud | Out of scope for v1 core; future optional sync must be additive and consent-based |
| Status | Normative engineering proposal; names and constraints should be treated as the v1 schema baseline |

## How to read this document

This specification is intentionally explicit. SQL types, nullability, defaults, uniqueness, foreign keys, delete behaviour, indexes, state transitions, and data ownership are described so implementation does not depend on guesswork. Where SQLite lacks a native type (for example UUID or timestamp-with-time-zone), MILO uses documented canonical encodings.

## Normative language

**MUST** means required for correctness or security. **SHOULD** means recommended unless a documented reason exists. **MAY** means optional. All timestamps are UTC ISO-8601 strings unless a field explicitly stores a local wall-clock value or an IANA/Windows time-zone identifier.

## Contents


1. Architecture and data ownership
2. Identity and authentication model
3. Data storage layout and database configuration
4. Entity relationship overview
5. Complete table catalogue
6. Relationships, deletion rules, and lifecycle
7. Authentication and session flows
8. File staging and filesystem ownership
9. Backend API / IPC contracts
10. Security, privacy, and threat model
11. Migrations, backup, restore, and portability
12. Retention, cleanup, and observability
13. Indexes, constraints, and query patterns
14. Validation, testing, and acceptance criteria
15. Implementation sequence and open decisions

Appendix A. Consolidated SQLite DDL

Appendix B. Enum and state definitions

Appendix C. Reference documentation

# 1. Architecture and data ownership

## 1.1 High-level architecture

MILO is a desktop application, not a hosted web service. The frontend renders the floating island and feature panels. It does not directly open the SQLite file, access arbitrary paths, or hold secrets. The Rust backend owns persistence, OS integration, scheduling, filesystem operations, and authorization checks. Frontend-to-backend communication occurs through narrowly scoped Tauri commands.

| Layer | Responsibility | Must not do |
|---|---|---|
| React / WebView | Render UI, collect user input, display validated DTOs, manage ephemeral view state | Direct SQL, arbitrary filesystem access, storing credentials, making authorization decisions |
| Tauri IPC | Typed command boundary; serialize requests/responses; reject invalid payloads | Expose broad generic shell or filesystem commands |
| Rust domain services | Business rules, validation, authorization, transactions, scheduling, OS adapters | Trust frontend-supplied ownership IDs without checking them |
| Repository / SQLite | Persist domain entities and indexes; enforce FK/unique/check constraints | Store access tokens, passwords, or raw secrets |
| Windows adapters | DPAPI, credential vault, media session, power/network/brightness, native drag/drop | Send private user content to remote services without consent |
| Optional future sync service | Replicate explicitly opted-in data across devices | Become a prerequisite for core local features |

## 1.2 Ownership principles

• **Local ownership:** each installation profile owns its local records. A profile is not a remote account.

• **Windows boundary:** the database directory is scoped to the current Windows user and protected by normal user-profile ACLs.

• **Least privilege:** each command is feature-specific; avoid generic `execute_sql`, `read_any_file`, or `run_command` IPC.

• **Data minimisation:** persist only what is needed to provide a feature. Avoid collecting telemetry by default.

• **Explicit file custody:** file staging records a reference to a user-owned source path. MILO must not move, rename, overwrite, or delete the original.

• **Offline by default:** local features continue working without network access.

## 1.3 What counts as user data?

| Data class | Examples | Storage |
|---|---|---|
| User-created content | Tasks, reminders, notes, alarm labels | SQLite |
| Preferences | Theme, deck order, sound settings, layout | SQLite |
| Operational state | Timer state, snooze state, last selected section | SQLite; runtime state reconciled on launch |
| OS-derived state | Battery, volume, brightness, network status | Mostly ephemeral; cache only when useful |
| File references | Display name, source path, file size, timestamps | SQLite; paths treated as sensitive metadata |
| Secrets | Future refresh tokens, API credentials, encryption keys | DPAPI / Credential Manager, never SQLite plaintext |
| Media metadata | Current track title/artist/artwork cache reference | Ephemeral by default; only cache when user opts in |
| Diagnostics | Crash IDs, error codes, app version | Local bounded log; upload only with explicit opt-in |

Do not build a generic cloud user database for v1. It would add account recovery, breach surface, legal obligations, and operational cost without improving the core offline desktop experience.

# 2. Identity and authentication model

MILO needs a clear identity boundary, but that does not mean it needs an email/password account. Authentication answers “who is operating this Windows session?” Authorization answers “which MILO actions may this process perform?”

## 2.1 Identity concepts

| Concept | Definition | Persisted? |
|---|---|---|
| Windows principal | The current Windows user/SID under which the process runs. | Do not store raw credentials. A non-secret SID hash may be recorded for diagnostics/ownership validation. |
| Installation | One installed copy / app instance lineage, represented by a random UUID. | Yes: app_installation |
| Local profile | A local MILO data namespace owned by the current Windows user. | Yes: local_profile |
| App session | A runtime session created when MILO starts and closed on orderly shutdown. | Yes: app_session |
| Authentication method | Local Windows-session presence; optional future Windows Hello re-auth for sensitive actions. | Yes: auth_event metadata only; no biometric templates |
| Cloud account | Optional future identity for cross-device sync, not required for local operation. | No v1 account tables |

## 2.2 First-run identity creation

• Installer creates the application package; it does not create a user account.

• On first launch, Rust obtains the current Windows user context and resolves the per-user app data directory.

• Generate a cryptographically random installation UUID and a separate local profile UUID.

• Create the SQLite database with restrictive per-user filesystem permissions; initialise schema migrations and seed default preferences.

• Record a `local_profile` row with `auth_mode = 'windows_session'` and `is_default = 1`.

• Create an `app_session` row with a random session UUID, start time, app version, and process/session metadata.

• Show the user a concise privacy notice explaining where local data lives and how to export or delete it.

## 2.3 Normal launch authentication

For v1, the Windows logon session is the primary authentication event. If the user can launch MILO under their Windows account and access the per-user data directory, MILO opens that local profile. MILO does not ask for another password. This is not a claim that the app has independently verified the human at the keyboard; it relies on the operating system's logged-in user boundary.

```
Windows logon
   |
   v
MILO process starts under current Windows token
   |
   v
Resolve current user data directory + Windows SID
   |
   v
Open profile database (user-scoped ACL)
   |
   +--> DB integrity / migration checks
   |
   v
Load local_profile + installation record
   |
   v
Create app_session + audit auth_event(startup)
   |
   v
App ready; feature commands apply profile/session checks
```

## 2.4 Optional sensitive-action re-authentication

A future release MAY require Windows Hello or an OS credential prompt before exporting all data, revealing a protected secret, or enabling cloud sync. Use Windows-provided verification; do not capture or store a PIN, password, fingerprint, or face template. Re-authentication should return a short-lived in-memory authorization grant scoped to one action, not a reusable frontend token.

| Action | v1 default | Future optional step |
|---|---|---|
| Open MILO / view local tasks | Windows session is sufficient | None |
| Create/edit local reminders | No extra prompt | None |
| Read a staged file's metadata | Normal file access checks | None |

| Action | v1 default | Future optional step |
|---|---|---|
| Export all local data | Explicit confirmation dialog | Windows Hello / OS confirmation may be offered |
| Enable cloud sync | Not available in v1 | Fresh OS re-auth + explicit consent |
| Reveal stored integration credential | No such v1 feature | Re-auth + one-time scoped grant |

## 2.5 Passwords, accounts, and recovery

**Never create a local password table for v1.** MILO must not duplicate Windows authentication, hash a user password, store password hints, or implement password reset. If a future cloud account is introduced, it must be a separate identity service with a separate threat model.

## 2.6 Future cloud authentication boundary

If multi-device sync becomes a product requirement, add a remote account service outside the local schema. The local profile remains usable offline. Cloud identity is linked to a local profile only after explicit user consent. Use an OAuth 2.1 / OpenID Connect authorization-code flow with PKCE in the system browser where supported; validate issuer, audience, nonce, state, signature, expiry, and redirect URI server-side. Store refresh tokens only in Windows Credential Manager or DPAPI-protected storage. Keep access tokens in memory where possible. Never place bearer tokens in SQLite, logs, URLs, crash reports, or frontend persistent storage.

| Future entity | Purpose | Why excluded from v1 |
|---|---|---|
| remote_account_link | Map local profile to remote subject / tenant | No online identity required |
| sync_device | Represent this device in the sync service | No sync service |
| sync_cursor | Track replication progress | No replication |
| sync_conflict | Store conflict resolution metadata | No replicated state |
| auth_event | Local audit of auth-related transitions | Local table included, but no remote account credentials |

# 3. Data storage layout and database configuration

## 3.1 Windows storage location

Recommended database path: `%LOCALAPPDATA%\MILO\data\milo.sqlite3`. The precise path should be resolved through the OS/app-data API, not hardcoded from a guessed username. Logs belong under `%LOCALAPPDATA%\MILO\logs\`; exports under a user-selected destination; temporary working files under a dedicated cache/temp directory.

```
%LOCALAPPDATA%\MILO\
  data\
    milo.sqlite3
    milo.sqlite3-wal
    milo.sqlite3-shm
    backups\
  cache\
    artwork\
    thumbnails\
    temp\
  logs\
    milo.log
  exports\                # optional; user-visible exports only
```

## 3.2 Database engine and settings

| Setting | Recommendation | Reason |
|---|---|---|
| Engine | SQLite 3; bundled/managed by Rust dependency | Embedded, offline, reliable local persistence |
| Journal | WAL mode | Concurrent reads with a single writer; durable transactional workflow |
| Foreign keys | PRAGMA foreign_keys = ON on every connection | SQLite FK enforcement is connection-scoped |
| Synchronous | FULL for high-integrity user content; evaluate NORMAL only with documented trade-off | Avoid sacrificing user-created data for marginal throughput |
| Busy timeout | 5,000 ms baseline, configurable | Graceful handling of brief writer contention |
| Encoding | UTF-8 | Consistent text storage |
| IDs | UUIDv4 strings, lowercase canonical form | Globally unique and portable |
| Timestamps | UTC RFC 3339 / ISO-8601 text with `Z` | Unambiguous ordering and portability |
| Booleans | INTEGER 0/1 with CHECK | SQLite has no separate boolean storage class |
| JSON | TEXT with JSON validity checks where available | Flexible low-volume metadata only; core queryable fields stay columns |
| Schema version | PRAGMA user_version + schema_migrations table | Deterministic migrations |

## 3.3 Connection and transaction policy

• Use one writer queue per application process. Read operations may use independent read-only connections.

• Every multi-row domain operation must run inside one transaction.

• Use prepared statements and parameter binding exclusively.

• Never hold a transaction open while awaiting UI input, OS prompts, network requests, or file copy operations.

• Use `BEGIN IMMEDIATE` for workflows that must reserve a state transition before updating related rows.

• Run `PRAGMA foreign_key_check` and an integrity check during diagnostics or controlled maintenance, not on every UI interaction.

## 3.4 SQLite security caveat

SQLite does not encrypt database content by default. The v1 baseline relies on per-user Windows filesystem ACLs and Windows account protection for the database file. If MILO later requires encryption at rest independent of the Windows profile, choose a maintained SQLite encryption solution and document key creation, rotation, backup, recovery, and migration. Do not invent custom encryption or store a database key beside the database.

WAL mode creates companion `-wal` and `-shm` files. Backup and restore must account for these correctly; do not copy only the main database file while it is live.

# 4. Entity relationship overview

The data model is grouped into bounded domains. Each table belongs to one local profile unless explicitly installation-wide. The schema uses `local_profile_id` on user-owned rows so future multi-profile support does not require rewriting every feature table.

```
app_installation 1---N local_profile 1---N app_session
                         |
                         +---1 profile_preferences
                         +---N deck_sections
                         +---N tasks 1---N task_checklist_items
                         |           +---N task_tags N---1 tags
                         +---N reminders 1---N reminder_occurrences
                         +---N timers 1---N timer_runs
                         +---N alarms
                         +---N staged_files 1---N staged_file_events
                         +---N media_favorites
                         +---N avatar_settings
                         +---N notifications
                         +---N quick_actions
                         +---N user_shortcuts
                         +---N app_events
                         +---N auth_events
```

```
                                  +---N export_jobs
                                  +---N permission_grants
```

## 4.1 Domain groups

| Group | Tables | Purpose |
|---|---|---|
| Installation / identity | app_installation, local_profile, app_session, auth_event | Establish local ownership and startup/shutdown lifecycle |
| Preferences / layout | profile_preferences, deck_sections, user_shortcuts | Store the user's chosen interface and interaction setup |
| Productivity | tasks, task_checklist_items, tags, task_tags | Tasks, checklist items, reusable tags |
| Scheduling | reminders, reminder_occurrences, timers, timer_runs, alarms | Recurring and one-off scheduling, timer persistence, alarm state |
| Files | staged_files, staged_file_events | Temporary references to user-selected files and staging actions |
| Media / avatar | media_favorites, avatar_settings | Optional media preferences and companion appearance settings |
| Notifications / actions | notifications, quick_actions | Notification queue/history and user-defined quick actions |
| Security / diagnostics | permission_grants, app_events, export_jobs, schema_migrations | Permission decisions, bounded diagnostics, data export, migrations |

# 5. Complete table catalogue

All tables below are part of the proposed v1 local schema. Column types are SQLite affinities; constraints shown are normative. UUIDs are stored as TEXT. Timestamps are UTC TEXT unless stated otherwise.

## 5.1 app_installation

One row per MILO installation lineage. This is not a user account and contains no credential.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4; installation identifier |
| created_at | TEXT NOT NULL | UTC creation timestamp |
| first_seen_version | TEXT NOT NULL | App version on first successful launch |
| last_seen_version | TEXT NOT NULL | Most recently launched app version |
| install_channel | TEXT NOT NULL | stable, beta, dev; CHECK enum |
| platform | TEXT NOT NULL | windows; reserved for future platform expansion |
| last_migration_at | TEXT NULL | Last successful migration timestamp |
| uninstall_requested_at | TEXT NULL | Set only if uninstall workflow records intent |

**Indexes:** UNIQUE(id); install_channel, created_at

**Rules:** Do not store device serial numbers, MAC addresses, Windows passwords, or raw Windows tokens.

## 5.2 local_profile

Local data namespace owned by the current Windows user. v1 normally has one default profile.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 local profile ID |
| installation_id | TEXT NOT NULL FK | References app_installation.id ON DELETE RESTRICT |
| display_name | TEXT NOT NULL | Optional local label; default 'My MILO' |
| windows_sid_hash | TEXT NULL | Optional keyed/non-reversible hash for ownership diagnostics; never raw SID unless justified |
| auth_mode | TEXT NOT NULL | windows_session |
| is_default | INTEGER NOT NULL | 0/1; CHECK |
| created_at | TEXT NOT NULL | UTC |
| updated_at | TEXT NOT NULL | UTC |
| deleted_at | TEXT NULL | Soft-delete marker for profile removal workflow |

**Indexes:** UNIQUE(installation_id, is_default) WHERE is_default = 1; installation_id

**Rules:** Never use display_name as an authorization key. Every query must scope by local_profile_id.

## 5.3 app_session

One runtime session per process launch. Used for clean shutdown, crash recovery, and diagnostics.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 session ID |
| local_profile_id | TEXT NOT NULL FK | References local_profile.id ON DELETE CASCADE |
| started_at | TEXT NOT NULL | UTC |
| ended_at | TEXT NULL | UTC; null means unclean/in-progress |
| app_version | TEXT NOT NULL | Version string |
| process_id | INTEGER NULL | OS process ID; diagnostic only |
| shutdown_reason | TEXT NULL | normal, crash_recovered, forced, update_restart |
| last_heartbeat_at | TEXT NULL | UTC; update at low frequency, not every UI frame |

**Indexes:** local_profile_id, started_at DESC; ended_at

**Rules:** On startup, mark stale sessions as crash-recovered only after a conservative process/heartbeat check.

## 5.4 auth_event

Minimal local audit trail for authentication and identity lifecycle events. It is not a credential store.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| local_profile_id | TEXT NULL FK | References local_profile.id ON DELETE SET NULL |
| session_id | TEXT NULL FK | References app_session.id ON DELETE SET NULL |
| event_type | TEXT NOT NULL | startup_accepted, profile_opened, reauth_requested, reauth_succeeded, reauth_failed, profile_locked, profile_deleted |
| auth_method | TEXT NOT NULL | windows_session, windows_hello, os_credential_prompt, none |
| result | TEXT NOT NULL | success, denied, error |
| occurred_at | TEXT NOT NULL | UTC |
| reason_code | TEXT NULL | Stable internal code; no secrets or raw OS error payloads |
| metadata_json | TEXT NULL | Allow-listed non-sensitive JSON only |

**Indexes:** local_profile_id, occurred_at DESC; event_type, occurred_at

**Rules:** Never record passwords, PINs, tokens, biometric results, file contents, or raw credential blobs.

## 5.5 profile_preferences

One row per profile for low-volume global preferences. Feature-specific settings should remain in their own tables.

| Column | Type | Rules / meaning |
|---|---|---|
| local_profile_id | TEXT PK/FK | References local_profile.id ON DELETE CASCADE |
| theme_mode | TEXT NOT NULL | system, dark, light |
| accent_key | TEXT NOT NULL | Named palette key; do not accept arbitrary CSS |
| launch_at_login | INTEGER NOT NULL | 0/1; default 0 |
| start_collapsed | INTEGER NOT NULL | 0/1; default 1 |
| close_behaviour | TEXT NOT NULL | hide_to_tray, exit |
| reduce_motion | INTEGER NOT NULL | 0/1 |
| sound_enabled | INTEGER NOT NULL | 0/1 |

| Column | Type | Rules / meaning |
|---|---|---|
| notification_enabled | INTEGER NOT NULL | 0/1 |
| week_start_day | INTEGER NOT NULL | 0-6; documented weekday convention |
| time_format | TEXT NOT NULL | system, 12h, 24h |
| date_format | TEXT NOT NULL | system or approved format key |
| timezone_id | TEXT NULL | Windows time-zone ID; null means follow OS |
| updated_at | TEXT NOT NULL | UTC |

**Indexes:** None beyond PK

**Rules:** Validate enum/range values in both Rust and SQLite CHECK constraints.

## 5.6 deck_sections

Stores the user's visible section order and per-section presentation preferences.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| local_profile_id | TEXT NOT NULL FK | References local_profile.id ON DELETE CASCADE |
| section_key | TEXT NOT NULL | overview, system, tasks, reminders, timers, alarms, files, music |
| sort_order | INTEGER NOT NULL | Zero-based display order |
| is_visible | INTEGER NOT NULL | 0/1 |
| is_pinned | INTEGER NOT NULL | 0/1 |
| collapsed_by_default | INTEGER NOT NULL | 0/1 |
| config_json | TEXT NULL | Allow-listed presentation-only options |
| created_at | TEXT NOT NULL | UTC |
| updated_at | TEXT NOT NULL | UTC |

**Indexes:** UNIQUE(local_profile_id, section_key); local_profile_id, sort_order

**Rules:** Do not store executable expressions, scripts, or arbitrary component names in config_json.

## 5.7 user_shortcuts

User-configurable keyboard shortcuts / quick launch mappings. Global shortcuts require OS registration and conflict handling.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| local_profile_id | TEXT NOT NULL FK | References local_profile.id ON DELETE CASCADE |
| action_key | TEXT NOT NULL | Allow-listed action identifier |
| accelerator | TEXT NOT NULL | Canonical shortcut representation |
| is_enabled | INTEGER NOT NULL | 0/1 |
| scope | TEXT NOT NULL | in_app, global |
| created_at | TEXT NOT NULL | UTC |
| updated_at | TEXT NOT NULL | UTC |

**Indexes:** UNIQUE(local_profile_id, action_key); UNIQUE(local_profile_id, accelerator) WHERE is_enabled = 1

**Rules:** Validate shortcuts in Rust and gracefully handle OS registration failures.

## 5.8 tasks

User-created task records. Completed tasks remain available until the user deletes or archives them.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| local_profile_id | TEXT NOT NULL FK | References local_profile.id ON DELETE CASCADE |
| title | TEXT NOT NULL | 1-500 chars after trimming |
| description | TEXT NULL | Plain text or sanitised Markdown; bounded length |
| status | TEXT NOT NULL | open, in_progress, completed, archived |
| priority | INTEGER NOT NULL | 0 none, 1 low, 2 normal, 3 high, 4 urgent |
| due_at | TEXT NULL | UTC instant; null means no due date |
| due_timezone_id | TEXT NULL | Zone used to interpret a user-entered local due time |
| completed_at | TEXT NULL | UTC; set when status becomes completed |
| sort_order | INTEGER NOT NULL | User ordering within list |
| created_at | TEXT NOT NULL | UTC |
| updated_at | TEXT NOT NULL | UTC |
| deleted_at | TEXT NULL | Soft delete; purge after retention policy |

**Indexes:** local_profile_id, status, sort_order; local_profile_id, due_at WHERE deleted_at IS NULL; local_profile_id, updated_at DESC

**Rules:** Use optimistic concurrency via updated_at or a version integer in update commands.

## 5.9 task_checklist_items

Ordered child checklist items for a task.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| task_id | TEXT NOT NULL FK | References tasks.id ON DELETE CASCADE |
| title | TEXT NOT NULL | 1-300 chars |
| is_completed | INTEGER NOT NULL | 0/1 |
| sort_order | INTEGER NOT NULL | Display order |
| created_at | TEXT NOT NULL | UTC |
| updated_at | TEXT NOT NULL | UTC |

**Indexes:** task_id, sort_order

**Rules:** Checklist completion does not automatically complete the parent task unless the product explicitly adopts that rule.

## 5.10 tags

Reusable profile-scoped tags.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| local_profile_id | TEXT NOT NULL FK | References local_profile.id ON DELETE CASCADE |
| name | TEXT NOT NULL | Trimmed; case-insensitive uniqueness per profile |
| colour_key | TEXT NULL | Approved palette token |
| created_at | TEXT NOT NULL | UTC |

**Indexes:** UNIQUE(local_profile_id, name COLLATE NOCASE)

**Rules:** Deleting a tag removes task_tags links, not tasks.

## 5.11 task_tags

Many-to-many join between tasks and tags.

| Column | Type | Rules / meaning |
|---|---|---|
| task_id | TEXT NOT NULL FK | References tasks.id ON DELETE CASCADE |
| tag_id | TEXT NOT NULL FK | References tags.id ON DELETE CASCADE |
| created_at | TEXT NOT NULL | UTC |

**Indexes:** PRIMARY KEY(task_id, tag_id)

**Rules:** Both referenced rows must belong to the same local_profile_id; enforce in repository transaction or via composite keys/triggers.

## 5.12 reminders

Reminder definitions. A reminder can be one-off or recurring; each firing is materialised as an occurrence.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| local_profile_id | TEXT NOT NULL FK | References local_profile.id ON DELETE CASCADE |
| title | TEXT NOT NULL | 1-300 chars |
| message | TEXT NULL | Optional body |
| schedule_type | TEXT NOT NULL | once, daily, weekly, monthly, custom |
| due_at | TEXT NULL | UTC for one-off reminders |
| local_time | TEXT NULL | HH:MM:SS wall-clock time for recurring reminders |
| timezone_id | TEXT NULL | Windows time-zone ID for recurring local schedule |
| recurrence_json | TEXT NULL | Validated recurrence object; no arbitrary code |
| is_enabled | INTEGER NOT NULL | 0/1 |
| sound_key | TEXT NULL | Allow-listed sound asset key |
| created_at | TEXT NOT NULL | UTC |
| updated_at | TEXT NOT NULL | UTC |
| deleted_at | TEXT NULL | Soft delete |

**Indexes:** local_profile_id, is_enabled; local_profile_id, due_at WHERE due_at IS NOT NULL

**Rules:** A reminder definition is not the same as a firing; use reminder_occurrences for each scheduled instance.

## 5.13 reminder_occurrences

Concrete scheduled and delivered instances of a reminder. Supports snooze, dismissal, and missed-fire recovery.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| reminder_id | TEXT NOT NULL FK | References reminders.id ON DELETE CASCADE |
| scheduled_for | TEXT NOT NULL | UTC instant |
| status | TEXT NOT NULL | scheduled, firing, delivered, snoozed, dismissed, missed, cancelled |
| delivered_at | TEXT NULL | UTC |
| snoozed_until | TEXT NULL | UTC |
| dismissed_at | TEXT NULL | UTC |
| attempt_count | INTEGER NOT NULL | Non-negative |
| last_error_code | TEXT NULL | Stable error code |
| created_at | TEXT NOT NULL | UTC |
| updated_at | TEXT NOT NULL | UTC |

**Indexes:** UNIQUE(reminder_id, scheduled_for); status, scheduled_for; reminder_id, scheduled_for DESC

**Rules:** Use idempotent occurrence generation to prevent duplicate notifications after crash/restart.

## 5.14 timers

Saved timer definitions, including duration and repeat preference.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| local_profile_id | TEXT NOT NULL FK | References local_profile.id ON DELETE CASCADE |
| label | TEXT NULL | Optional user label |
| duration_ms | INTEGER NOT NULL | 1..604800000 (maximum seven days) |
| repeat_enabled | INTEGER NOT NULL | 0/1 |
| sound_key | TEXT NULL | Allow-listed sound key |
| sort_order | INTEGER NOT NULL | Display order |
| created_at | TEXT NOT NULL | UTC |
| updated_at | TEXT NOT NULL | UTC |

**Indexes:** local_profile_id, sort_order

**Rules:** Do not store a live monotonic clock value as the sole source of truth across restarts.

## 5.15 timer_runs

Each actual timer run. Persist wall-clock deadlines and pause state so the timer can recover after restart.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| timer_id | TEXT NOT NULL FK | References timers.id ON DELETE CASCADE |
| status | TEXT NOT NULL | running, paused, completed, cancelled |
| started_at | TEXT NOT NULL | UTC |
| deadline_at | TEXT NULL | UTC deadline while running |
| remaining_ms | INTEGER NOT NULL | Remaining duration when paused or last reconciled |
| paused_at | TEXT NULL | UTC |
| completed_at | TEXT NULL | UTC |
| last_notified_at | TEXT NULL | UTC; idempotency aid |
| created_at | TEXT NOT NULL | UTC |
| updated_at | TEXT NOT NULL | UTC |

**Indexes:** timer_id, created_at DESC; status, deadline_at WHERE status = 'running'

**Rules:** Only one active run per timer unless product deliberately supports parallel runs. Enforce with partial unique index if adopted.

## 5.16 alarms

Alarm definitions with weekday recurrence and snooze configuration.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| local_profile_id | TEXT NOT NULL FK | References local_profile.id ON DELETE CASCADE |
| label | TEXT NULL | Optional label |
| local_time | TEXT NOT NULL | HH:MM:SS local wall-clock time |
| timezone_id | TEXT NULL | Windows time-zone ID; null follows OS |
| repeat_mask | INTEGER NOT NULL | 7-bit weekday mask; 0 means one-time |
| one_time_date | TEXT NULL | YYYY-MM-DD for one-time alarm |
| is_enabled | INTEGER NOT NULL | 0/1 |

| Column | Type | Rules / meaning |
|---|---|---|
| snooze_minutes | INTEGER NOT NULL | 1..60 |
| sound_key | TEXT NULL | Allow-listed sound key |
| next_fire_at | TEXT NULL | Materialised UTC next occurrence |
| last_fired_at | TEXT NULL | UTC |
| created_at | TEXT NOT NULL | UTC |
| updated_at | TEXT NOT NULL | UTC |

**Indexes:** local_profile_id, is_enabled, next_fire_at

**Rules:** Recompute next_fire_at on time-zone or DST changes; define explicit behaviour for ambiguous/nonexistent local times.

## 5.17 staged_files

Metadata-only references to files the user places in MILO's temporary holding area.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| local_profile_id | TEXT NOT NULL FK | References local_profile.id ON DELETE CASCADE |
| source_path | TEXT NOT NULL | Absolute path selected/dropped by user; sensitive metadata |
| display_name | TEXT NOT NULL | Filename shown in UI |
| extension | TEXT NULL | Normalised extension without dot |
| mime_type | TEXT NULL | Best-effort type; never trust as proof of file safety |
| size_bytes | INTEGER NULL | Observed size at staging time |
| modified_at_source | TEXT NULL | Source file modified timestamp if available |
| content_hash | TEXT NULL | Optional SHA-256; compute only when useful/consented |
| status | TEXT NOT NULL | staged, missing, unavailable, removed_from_shelf, expired |
| staged_at | TEXT NOT NULL | UTC |
| last_verified_at | TEXT NULL | UTC |
| expires_at | TEXT NULL | UTC; optional automatic shelf expiry |
| removed_at | TEXT NULL | UTC |

**Indexes:** local_profile_id, status, staged_at DESC; local_profile_id, source_path

**Rules:** Staging MUST NOT copy/move/delete the original by default. Treat source_path as sensitive; do not include it in telemetry.

## 5.18 staged_file_events

Append-only local event history for file shelf actions.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| staged_file_id | TEXT NULL FK | References staged_files.id ON DELETE SET NULL |
| local_profile_id | TEXT NOT NULL FK | References local_profile.id ON DELETE CASCADE |
| event_type | TEXT NOT NULL | added, verified, dragged_out, copied_to_temp, removed, expired, source_missing |
| occurred_at | TEXT NOT NULL | UTC |
| result | TEXT NOT NULL | success, cancelled, failed |
| error_code | TEXT NULL | Stable error code |
| details_json | TEXT NULL | Allow-listed non-sensitive metadata |

**Indexes:** local_profile_id, occurred_at DESC; staged_file_id, occurred_at DESC

**Rules:** Do not store file contents or full drag payloads in this event table.

## 5.19 media_favorites

Optional user-pinned media items. Current playback state is ephemeral and should not be retained by default.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| local_profile_id | TEXT NOT NULL FK | References local_profile.id ON DELETE CASCADE |
| provider_key | TEXT NOT NULL | Windows media session / approved provider identifier |
| provider_item_id | TEXT NULL | Provider's stable item identifier if available |
| title | TEXT NOT NULL | Track / media title |
| artist | TEXT NULL | Artist or creator |
| album | TEXT NULL | Album / collection |
| artwork_cache_key | TEXT NULL | Relative cache key; not a remote URL with credentials |
| created_at | TEXT NOT NULL | UTC |

**Indexes:** local_profile_id, created_at DESC; UNIQUE(local_profile_id, provider_key, provider_item_id) WHERE provider_item_id IS NOT NULL

**Rules:** Only store favourites if user explicitly chooses to save them.

## 5.20 avatar_settings

Per-profile companion configuration; no personal biometric or inferred emotion data.

| Column | Type | Rules / meaning |
|---|---|---|
| local_profile_id | TEXT PK/FK | References local_profile.id ON DELETE CASCADE |
| avatar_key | TEXT NOT NULL | Bundled allow-listed avatar asset key |
| enabled | INTEGER NOT NULL | 0/1 |
| animation_level | TEXT NOT NULL | full, reduced, minimal, off |
| idle_behavior | TEXT NOT NULL | subtle, still |
| sound_reactions_enabled | INTEGER NOT NULL | 0/1 |
| last_selected_at | TEXT NOT NULL | UTC |
| updated_at | TEXT NOT NULL | UTC |

**Indexes:** None beyond PK

**Rules:** Avatar state must not be used to infer or store the user's mood, attention, or emotional state.

## 5.21 notifications

Local notification queue and recent history. Old rows are pruned according to retention rules.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| local_profile_id | TEXT NOT NULL FK | References local_profile.id ON DELETE CASCADE |
| source_type | TEXT NOT NULL | reminder, timer, alarm, system, app |
| source_id | TEXT NULL | Source record ID; no FK because source tables differ |
| title | TEXT NOT NULL | Display title |
| body | TEXT NULL | Display body |
| status | TEXT NOT NULL | pending, shown, dismissed, actioned, failed |
| scheduled_at | TEXT NULL | UTC |
| shown_at | TEXT NULL | UTC |
| dismissed_at | TEXT NULL | UTC |
| action_key | TEXT NULL | Allow-listed action identifier |

| Column | Type | Rules / meaning |
|---|---|---|
| created_at | TEXT NOT NULL | UTC |

**Indexes:** local_profile_id, status, scheduled_at; local_profile_id, created_at DESC

**Rules:** Avoid storing sensitive content in OS notification history when Windows may display it on a lock screen.

## 5.22 quick_actions

User-configured actions surfaced in MILO's quick action panel.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| local_profile_id | TEXT NOT NULL FK | References local_profile.id ON DELETE CASCADE |
| action_key | TEXT NOT NULL | Allow-listed action identifier |
| label_override | TEXT NULL | Optional UI label |
| sort_order | INTEGER NOT NULL | Display order |
| is_enabled | INTEGER NOT NULL | 0/1 |
| config_json | TEXT NULL | Validated action parameters only |
| created_at | TEXT NOT NULL | UTC |
| updated_at | TEXT NOT NULL | UTC |

**Indexes:** local_profile_id, sort_order; UNIQUE(local_profile_id, action_key)

**Rules:** Never permit a quick action to store or execute arbitrary shell commands or scripts.

## 5.23 permission_grants

Records user consent for optional MILO features or data access. This is an application consent ledger, not a substitute for OS ACLs.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| local_profile_id | TEXT NOT NULL FK | References local_profile.id ON DELETE CASCADE |
| permission_key | TEXT NOT NULL | e.g. launch_at_login, notifications, optional_telemetry, media_metadata_cache |
| decision | TEXT NOT NULL | granted, denied, revoked |
| scope_json | TEXT NULL | Narrow, validated scope |
| decided_at | TEXT NOT NULL | UTC |
| expires_at | TEXT NULL | UTC |
| revoked_at | TEXT NULL | UTC |
| source | TEXT NOT NULL | onboarding, settings, feature_prompt |

**Indexes:** local_profile_id, permission_key, decided_at DESC

**Rules:** A permission change must be applied to the OS integration and recorded transactionally where feasible.

## 5.24 export_jobs

Tracks explicit user-requested data exports. Does not store the exported content in the database.

| Column | Type | Rules / meaning |
|---|---|---|
| id | TEXT PK | UUIDv4 |
| local_profile_id | TEXT NOT NULL FK | References local_profile.id ON DELETE CASCADE |
| format | TEXT NOT NULL | json, csv, zip |
| status | TEXT NOT NULL | queued, running, completed, failed, cancelled |
| destination_path | TEXT NULL | User-selected destination; sensitive metadata |

| Column | Type | Rules / meaning |
|---|---|---|
| requested_at | TEXT NOT NULL | UTC |
| completed_at | TEXT NULL | UTC |
| expires_at | TEXT NULL | UTC; cleanup metadata only |
| error_code | TEXT NULL | Stable code |
| includes_file_paths | INTEGER NOT NULL | 0/1; export option must be explicit |

**Indexes:** local_profile_id, requested_at DESC

**Rules:** Default export excludes absolute file paths, diagnostic logs, and secrets.

## 5.25 app_events

Bounded local diagnostics for application lifecycle and feature failures. This is not a behavioural analytics pipeline.

| Column | Type | Rules / meaning |
|---|---|---|
| id | INTEGER PK AUTOINCREMENT | Local monotonic row identifier |
| session_id | TEXT NULL FK | References app_session.id ON DELETE SET NULL |
| local_profile_id | TEXT NULL FK | References local_profile.id ON DELETE SET NULL |
| event_name | TEXT NOT NULL | Allow-listed event name |
| severity | TEXT NOT NULL | debug, info, warning, error |
| occurred_at | TEXT NOT NULL | UTC |
| error_code | TEXT NULL | Stable internal code |
| component | TEXT NULL | Backend component identifier |
| duration_ms | INTEGER NULL | Optional measured duration |
| details_json | TEXT NULL | Allow-listed, scrubbed, size-limited metadata |

**Indexes:** occurred_at DESC; severity, occurred_at DESC; session_id, occurred_at

**Rules:** Never log user task text, reminder body, file path, access token, raw command payload, or full OS error object.

## 5.26 schema_migrations

Records successful schema migrations and their checksums.

| Column | Type | Rules / meaning |
|---|---|---|
| version | INTEGER PK | Monotonically increasing migration number |
| name | TEXT NOT NULL | Stable migration name |
| checksum | TEXT NOT NULL | SHA-256 of migration source |
| applied_at | TEXT NOT NULL | UTC |
| execution_ms | INTEGER NULL | Duration |
| app_version | TEXT NOT NULL | App version that applied migration |

**Indexes:** UNIQUE(name); applied_at DESC

**Rules:** Never edit a migration after release; add a new forward migration.

# 6. Relationships, deletion rules, and lifecycle

## 6.1 Relationship cardinalities

| Parent | Child | Cardinality | Delete behaviour |
|---|---|---|---|
| app_installation | local_profile | 1:N | RESTRICT; remove profile through explicit workflow first |
| local_profile | app_session | 1:N | CASCADE |
| local_profile | profile_preferences | 1:1 | CASCADE |
| local_profile | deck_sections | 1:N | CASCADE |
| local_profile | tasks | 1:N | CASCADE |
| tasks | task_checklist_items | 1:N | CASCADE |
| tasks | task_tags | 1:N | CASCADE |
| tags | task_tags | 1:N | CASCADE |
| local_profile | reminders | 1:N | CASCADE |
| reminders | reminder_occurrences | 1:N | CASCADE |
| local_profile | timers | 1:N | CASCADE |
| timers | timer_runs | 1:N | CASCADE |
| local_profile | alarms | 1:N | CASCADE |
| local_profile | staged_files | 1:N | CASCADE |
| staged_files | staged_file_events | 1:N | SET NULL on file delete; profile still owns event |
| local_profile | notifications | 1:N | CASCADE |
| local_profile | permission_grants | 1:N | CASCADE |

## 6.2 Soft delete versus hard delete

Use soft delete for user-facing content where accidental deletion recovery is useful (tasks, reminders, staged shelf entries). Use hard delete for ephemeral history and join rows where retention is bounded. Profile deletion is a deliberate multi-step workflow: export prompt, confirmation, stop scheduler, remove OS registrations, close DB, delete profile-owned rows transactionally, then remove cache files. The app must never delete source files referenced by `staged_files`.

## 6.3 Referential integrity across profile boundaries

A task and tag must belong to the same profile. A simple foreign key from `task_tags` cannot guarantee this by itself when only IDs are stored. Recommended implementation: add `local_profile_id` to the join table and use composite unique keys `(id, local_profile_id)` on parent tables plus composite foreign keys, or enforce the invariant in one repository transaction and add triggers. Prefer composite foreign keys for strongest database-level enforcement.

## 6.4 Scheduler state reconciliation

• At startup, load enabled reminders and alarms, calculate next due instants, and reconcile materialised occurrences.

• For running timers, compare persisted `deadline_at` with current UTC; do not rely on process uptime or a monotonic counter persisted across reboots.

• On sleep/resume and time-zone change, recompute future scheduling and update `next_fire_at`.

• Use idempotency keys / unique constraints to avoid duplicate reminder occurrences and notifications.

• Never block the UI thread while reconciling scheduler state.

# 7. Authentication and session flows

## 7.1 First-run sequence

```
[Installer] -> [MILO first launch]
                    |
                    v
               Resolve Windows user context
                    |
                    v
               Resolve %LOCALAPPDATA% path
                    |
                    v
               Create installation UUID
                    |
                    v
               Create SQLite + apply migrations
                    |
                    v
               Create local_profile + defaults
                    |
                    v
               Create app_session + auth_event
                    |
                    v
               Display local-data/privacy explanation
                    |
                    v
                  READY
```

## 7.2 Existing profile launch

```
Process starts as current Windows user
  -> resolve expected per-user DB path
  -> verify directory ownership / access
  -> open DB with foreign keys enabled
  -> run migration preflight
  -> verify profile belongs to current installation
  -> recover interrupted session / scheduler state
  -> insert app_session
  -> append auth_event(startup_accepted)
  -> load preferences and sections
  -> READY
```

## 7.3 Profile access denied / Windows account changed

If the database directory cannot be opened, MILO must fail closed for that data profile. Do not create an empty replacement database over the existing path. Present a clear recovery path: check Windows account, permissions, backup, or restore. If a user copies the database to another Windows account, normal ACLs and any DPAPI-protected secrets may prevent access; that is expected and should be explained.

## 7.4 Session lifecycle

| Event | Backend action | Persistence |
|---|---|---|
| Startup | Create app_session; append auth_event | One session row + event |
| Normal operation | Update heartbeat at a low cadence | last_heartbeat_at |
| User closes window to tray | Keep process/session active | No session end |
| User exits app | Stop scheduler, unregister shortcuts, flush DB, close session | ended_at + shutdown_reason |
| Crash / forced termination | Next launch detects stale session and reconciles | shutdown_reason=crash_recovered |
| Windows sign-out / shutdown | Best-effort graceful close; startup recovery remains authoritative | Session may be left open |

## 7.5 Future cloud login sequence (not v1)

If a cloud feature is approved, the login flow should be separate from local launch. Local functionality remains available if login fails or the service is offline.

```
User chooses "Enable sync"
  -> show exact data categories to be synced
  -> explain destination, retention, and revocation
  -> fresh OS re-authentication (optional but recommended)
  -> launch system browser to identity provider
  -> Authorization Code + PKCE
```

```
  -> validate state / nonce / issuer / audience / signature
  -> exchange code at trusted backend
  -> store refresh token in Windows Credential Manager / DPAPI
  -> keep short-lived access token in memory
  -> link remote subject to local_profile only after consent
  -> sync only selected entities
```

### 7.6 Authentication threat boundaries

• A Windows session is not equivalent to verifying the person is physically present.

• A local administrator or malware running as the same user may access data; local app architecture cannot fully defend against a compromised OS account.

• Do not expose a frontend command that returns DPAPI plaintext or a long-lived token.

• Re-auth grants must be scoped, short-lived, and validated in Rust for the requested action.

• Do not store Windows Hello templates, fingerprints, face data, or PINs.

## 8. File staging and filesystem ownership

### 8.1 Data model intent

The MILO file shelf is a staging convenience, not a file manager. The default operation stores a source path and metadata in `staged_files`. It does not duplicate the file into MILO storage. When the user drags a staged item into another app, the backend validates the source still exists and initiates a native Windows drag-out flow.

### 8.2 Staging sequence

```
User selects/drops file
  -> validate supported path and access
  -> inspect metadata (name, extension, size, modified time)
  -> optionally compute hash only if justified
  -> insert staged_files row + staged_file_events(added)
  -> show item in shelf
  -> source file remains at original path
```

### 8.3 Drag-out sequence

• Revalidate path existence and read access immediately before drag-out.

• Use native Windows OLE drag-and-drop from the Rust/native layer; do not fake drag-out by exposing arbitrary local paths to the WebView.

• Record outcome only after the OS drag session completes or is cancelled.

• If source has moved or disappeared, mark status `missing` and offer remove/reselect.

• Never delete, rename, or move the source after a successful drag.

### 8.4 Optional temporary copy

If a later feature requires a stable copy (for example, user explicitly chooses 'Make a temporary copy'), store it under a MILO-owned temp directory and record the managed copy path separately in a future `staged_file_copies` table. The UI must distinguish 'reference to original' from 'MILO temporary copy'. Add expiry, quota, cleanup and restore semantics before shipping this feature.

### 8.5 File privacy

Absolute paths can reveal names, projects, clients, and folder structures. Exclude them from analytics, crash logs, notification bodies, and default exports. If file paths are included in an export, require an explicit checkbox/confirmation.

## 9. Backend API / IPC contracts

IPC commands are the public interface between React and Rust. The frontend sends typed input DTOs; Rust validates, checks the active profile, executes domain logic, and returns typed response DTOs. The database schema must not leak directly to the frontend.

## 9.1 Command naming

| Domain | Representative commands |
|---|---|
| App / profile | app_get_status, profile_get_current, profile_export, profile_delete_confirmed |
| Preferences | preferences_get, preferences_update, deck_sections_reorder |
| Tasks | task_list, task_create, task_update, task_complete, task_archive, task_delete |
| Reminders | reminder_list, reminder_create, reminder_update, reminder_snooze, reminder_dismiss |
| Timers | timer_list, timer_start, timer_pause, timer_resume, timer_cancel |
| Alarms | alarm_list, alarm_create, alarm_update, alarm_toggle, alarm_snooze |
| Files | staged_file_add, staged_file_list, staged_file_remove, staged_file_refresh, staged_file_drag_out |
| Media | media_get_current_session, media_play_pause, media_next, media_previous, media_favorite_add |
| Settings | settings_get, settings_update, permission_request, permission_revoke |
| Diagnostics | diagnostics_get_status, diagnostics_export_opt_in |

## 9.2 Command request rules

• Every request must include only the fields required for that operation; do not accept raw SQL, table names, or file paths for arbitrary backend use.

• The active local profile is derived from trusted backend session state. Do not trust a `local_profile_id` sent by the frontend.

• For update commands, require entity ID plus a version/updated_at precondition to prevent stale writes.

• All text is length-limited and normalised in Rust; UI validation is convenience only.

• Return stable error codes, not raw SQL errors, stack traces, or OS error structures.

## 9.3 Example DTOs

```
TaskCreateRequest {
  title: String,                 // required, trimmed, <= 500 chars
  description: Option<String>,   // <= configured max
  priority: u8,                  // 0..4
  due_at: Option<DateTime<Utc>>
}

TaskDto {
  id: Uuid,
  title: String,
  description: Option<String>,
  status: TaskStatus,
  priority: u8,
  due_at: Option<DateTime<Utc>>,
  completed_at: Option<DateTime<Utc>>,
  created_at: DateTime<Utc>,
  updated_at: DateTime<Utc>
}

StagedFileAddRequest {
  selected_path: PathBuf         // accepted only from a trusted picker/drop path
}

StagedFileDto {
  id: Uuid,
  display_name: String,
  extension: Option<String>,
  size_bytes: Option<u64>,
  status: StagedFileStatus,
  staged_at: DateTime<Utc>
  // source_path deliberately omitted from ordinary UI DTO
}
```

## 9.4 Error envelope

```
{
  "code": "TASK_NOT_FOUND",
  "message": "That task is no longer available.",
  "retryable": false,
  "correlation_id": "uuid"
}
```

Error messages are safe for display. Internal logs may include a correlation ID and allow-listed technical context, but never user content or secrets.

# 10. Security, privacy, and threat model

## 10.1 Security controls by layer

| Threat / surface | Required control |
|---|---|
| Compromised WebView / XSS | Strict CSP; no remote content in privileged window; narrow Tauri capabilities; no arbitrary IPC |
| Over-privileged IPC | Per-window capabilities; command allow-list; Rust-side authorization and input validation |
| Database theft from another account | Per-user directory ACLs; optional future DB encryption; no secrets in DB |
| Secret exposure | DPAPI user scope / Credential Manager; no plaintext logs; short-lived in-memory access tokens |
| SQL injection | Prepared statements; no dynamic SQL from user input; allow-list sort/filter keys |
| Path traversal / file abuse | Canonicalise and validate paths; accept user picker/drop references; no arbitrary write/delete commands |
| Data corruption | Transactions; WAL-aware backup; migrations with checksums; integrity checks |
| Accidental data loss | User-controlled export; clear deletion confirmation; safe migration rollback/backup |
| Telemetry leakage | Off by default; scrubbed allow-listed events; explicit consent |
| Supply-chain compromise | Pinned dependencies, lockfiles, signed releases, reproducible build practices where feasible |

## 10.2 Secret storage rules

• DPAPI `CurrentUser` scope is appropriate for secrets bound to the current Windows user. Do not use machine-wide protection for user-specific tokens.

• Windows Credential Manager may store application generic credentials where a credential target is appropriate.

• SQLite stores only a non-secret reference/key identifier, if required; never the secret value.

• No secrets in environment variables, command-line arguments, URLs, debug logs, frontend localStorage, or crash dumps by design.

• Secret deletion must remove the vault entry and clear in-memory copies as far as practical.

## 10.3 Tauri command permissions

Define explicit capabilities for the main island window and any auxiliary windows. Expose only required plugins and commands. Avoid broad filesystem permissions. Keep file picking, file staging, and native drag-out as separate privileged commands with narrowly scoped inputs. Tauri capabilities reduce the WebView's exposed surface, but they do not replace Rust-side validation.

## 10.4 Privacy defaults

| Feature | Default | User control |
|---|---|---|
| Local database | Enabled; required for local app | Export / delete profile |
| Cloud sync | Off / absent in v1 | Future opt-in per category |
| Telemetry | Off | Explicit opt-in |
| Crash report upload | Off | Per-report or global opt-in |
| Media metadata cache | Ephemeral | Optional favourite/pin |
| File hash | Not computed by default | Only compute when feature requires it |
| File path export | Excluded | Explicit include-paths option |
| Avatar | Bundled assets only | Disable / reduce motion |

# 11. Migrations, backup, restore, and portability

## 11.1 Migration policy

• Use numbered, immutable migration files: `0001_initial.sql`, `0002_add_alarm_timezone.sql`, etc.

• Before migration, ensure the database is closed to other writers and create a consistent backup.

• Run each migration in a transaction where SQLite permits; record checksum and applied version.

• If a migration fails, abort startup for that profile and preserve the original database.

• Never silently drop columns or user content. Destructive changes require a data-preserving migration and explicit release notes.

## 11.2 Backup policy

Use SQLite's online backup API or a correctly coordinated checkpoint/backup strategy. Do not copy the live main database file alone while WAL writes are active. Keep a small rolling set of local backups (for example, last 3 successful pre-migration snapshots) subject to disk quota and user-visible controls. Backups may contain personal data and must remain in the same user-protected directory.

## 11.3 Export format

| Format | Use | Contents |
|---|---|---|
| JSON | Full-fidelity backup / portability | Versioned manifest + profile preferences + tasks + schedules + app settings; omit secrets |
| CSV | Human-readable task export | Tasks and selected simple entities; relationships may be flattened |
| ZIP | Complete user-requested export | JSON/CSV plus optional user-selected attachments if future feature exists |

## 11.4 Restore

• Preview the source file and schema version before writing.

• Restore into a new local profile or explicit replace workflow; never overwrite the active profile without confirmation.

• Validate JSON structure, lengths, enum values, relationship IDs, and archive paths.

• Never import secrets or executable configuration.

• Rebuild scheduler materialisations after restore; do not blindly trust old `next_fire_at` values.

## 11.5 Portability

A plain database copy may be portable for non-secret local data, but Windows ACLs and DPAPI-protected secrets are not portable by design. The export format is the supported transfer mechanism. State this clearly in UI copy.

# 12. Retention, cleanup, and observability

| Data | Suggested retention | Cleanup behaviour |
|---|---|---|
| app_session | 90 days | Prune ended sessions; retain recent crash context |
| auth_event | 180 days | Prune by age; never store secret payloads |
| app_events | 30 days / bounded row cap | Delete oldest rows when size cap reached |
| notifications | 30 days | Prune dismissed/shown history |
| reminder_occurrences | 90 days | Retain recent history; delete old dismissed/missed occurrences |
| timer_runs | 90 days | Retain limited recent runs |
| staged_files | Until user removes / configured expiry | Expire shelf reference only; never delete source file |
| staged_file_events | 90 days | Prune old events; no path content in details |
| export_jobs | 30 days | Delete job metadata; user-selected exported file is not auto-deleted |
| backups | Rolling last 3 or quota policy | Delete oldest backups after verifying newest backup |

## 12.1 Diagnostics constraints

Diagnostics should answer: which component failed, when, with which stable error code, and whether retry is possible. Diagnostics should not capture what the user typed, what file they opened, which websites they visited, or what music they listened to. Keep an allow-list of event names and metadata fields.

## 12.2 Data lifecycle states

For entities with soft deletion, use `deleted_at` and filter active rows by `deleted_at IS NULL`. A purge job may permanently delete rows after the documented recovery window. Purge must respect foreign keys and never touch external source files.

# 13. Indexes, constraints, and query patterns

## 13.1 Required indexes

| Table | Index / constraint | Supports |
|---|---|---|
| local_profile | installation_id; unique default profile | Resolve current profile |
| app_session | profile + started_at DESC; ended_at | Session recovery / diagnostics |
| auth_event | profile + occurred_at DESC | Audit review |
| deck_sections | unique profile + section_key; profile + sort_order | Deck rendering |
| tasks | profile + status + sort_order; partial due index | Task list / due views |
| task_checklist_items | task_id + sort_order | Checklist render |
| tags | unique profile + case-insensitive name | Tag lookup |
| task_tags | PK task_id + tag_id | Task/tag joins |
| reminders | profile + enabled; due_at partial | Scheduler query |
| reminder_occurrences | unique reminder + scheduled_for; status + scheduled_for | Idempotent delivery |
| timer_runs | timer_id + created_at DESC; running deadline partial | Active timer recovery |
| alarms | profile + enabled + next_fire_at | Next alarm lookup |
| staged_files | profile + status + staged_at DESC | Shelf listing |
| notifications | profile + status + scheduled_at | Notification queue |
| app_events | occurred_at DESC; severity + occurred_at | Diagnostics |

## 13.2 Query patterns

```
-- Open tasks, oldest user ordering first
SELECT id, title, priority, due_at, status
FROM tasks
WHERE local_profile_id = ?1
  AND deleted_at IS NULL
  AND status IN ('open', 'in_progress')
ORDER BY sort_order ASC, created_at ASC;

-- Due enabled reminders
SELECT r.id, r.title, o.id, o.scheduled_for
FROM reminders r
JOIN reminder_occurrences o ON o.reminder_id = r.id
WHERE r.local_profile_id = ?1
  AND r.is_enabled = 1
  AND r.deleted_at IS NULL
  AND o.status = 'scheduled'
  AND o.scheduled_for <= ?2
ORDER BY o.scheduled_for ASC;

-- Active timer runs
SELECT tr.*, t.label, t.duration_ms
FROM timer_runs tr
JOIN timers t ON t.id = tr.timer_id
WHERE t.local_profile_id = ?1
  AND tr.status IN ('running', 'paused')
ORDER BY tr.created_at DESC;
```

## 13.3 Constraints

• All IDs are non-empty UUID strings; validate at service boundary.

• All user-owned rows reference a valid local profile directly or through a parent.

• Boolean columns have CHECK constraints limited to 0/1.

• Priority, status, schedule_type, auth_mode, and other enums use CHECK constraints.

• Text fields have application-level length bounds; SQLite CHECK length constraints should also be added for critical fields.

• No cross-profile relationship is permitted.

• Use partial unique indexes for active-only uniqueness where SQLite supports the required predicate.

# 14. Validation, testing, and acceptance criteria

## 14.1 Schema tests

• Fresh database creation produces the latest schema and all required indexes.

• Migration from every released schema version preserves user-created data.

• Foreign keys are enabled on every connection and `foreign_key_check` returns no rows.

• Invalid enum values, negative durations, malformed repeat masks, and duplicate unique values are rejected.

• Cross-profile task/tag links are rejected.

• Deleting a task cascades to checklist and join rows but does not delete unrelated tags.

## 14.2 Authentication tests

• First launch creates one installation, one default profile, one app session, and an auth event.

• Subsequent launch reuses the existing profile; it does not create a new profile each time.

• A different Windows user cannot silently open the prior user's data directory.

• If access is denied, MILO does not create an empty replacement database over the inaccessible file.

• Crash recovery closes stale sessions with a crash-recovered reason and restores scheduler state.

• No password, PIN, biometric template, token, or secret appears in SQLite or logs.

## 14.3 File staging tests

• Adding a file records metadata and leaves source path, content, and timestamps unchanged.

• Dragging out a file does not move or delete the source.

• Missing or inaccessible files are marked and handled gracefully.

• File paths never appear in telemetry or default exports.

• Removing an item from the shelf removes the reference only.

## 14.4 Scheduler tests

• Recurring reminders generate one occurrence per scheduled instant, including after restart.

• Duplicate scheduler invocations do not duplicate an occurrence or notification.

• DST transitions, time-zone changes, sleep/resume, and system clock changes are tested.

• A timer recovers correctly after app close, crash, sleep, and reboot.

## 14.5 Performance and reliability targets

| Operation | Target |
|---|---|
| Open DB / resolve profile | p95 under 150 ms on reference hardware, excluding migration |
| Read common dashboard data | p95 under 100 ms |
| Create/update task | p95 under 100 ms |
| Startup migration | Measured and bounded; show progress for long migrations |
| Background DB work | Never block UI rendering or animation thread |
| Diagnostics storage | Bounded by row count and disk quota |
| Recovery | No silent data loss; explicit actionable error on failed migration |

# 15. Implementation sequence and open decisions

## 15.1 Recommended build order

• Implement SQLite connection manager, WAL policy, foreign keys, transaction helpers, and migration runner.

• Implement app_installation, local_profile, app_session, auth_event, and first-run/launch recovery.

• Implement profile-scoped repository pattern and typed IPC request/response DTOs.

• Implement profile_preferences, deck_sections, and user_shortcuts.

• Implement tasks, checklist items, tags, and join integrity.

• Implement reminders, occurrences, timer definitions/runs, alarms, and scheduler reconciliation.

• Implement staged_files and staged_file_events with metadata-only source references.

• Implement notifications, quick actions, avatar settings, and media favourites.

• Implement export/restore, retention, backup, and diagnostics controls.

• Complete security review, permission/capability audit, migration tests, and release validation.

## 15.2 Open decisions to resolve before implementation

| Decision | Recommended baseline | When to revisit |
|---|---|---|
| Multiple MILO profiles per Windows user? | Schema supports it; UI exposes one default profile in v1 | If shared PC / separate workspaces requested |
| Database encryption at rest? | User-scoped Windows ACLs + DPAPI for secrets; evaluate SQLCipher only if threat model requires | Before storing high-sensitivity local content |
| Automatic backup frequency? | Pre-migration + bounded rolling snapshots; optional scheduled backup | Before public beta |
| Task recurrence? | Not in baseline tasks; use reminders/alarms first | If recurring task demand is validated |
| File temporary copies? | Not in v1 default; metadata-only references | Only with explicit user need and quota design |
| Cloud sync? | No v1 implementation | After privacy, conflict, and service architecture approval |
| Windows Hello re-auth? | Optional future sensitive-action guard | If export/sync threat model warrants it |

## 15.3 Non-negotiable implementation rules

• No mandatory MILO account for core local functionality.

• No passwords, tokens, biometric material, or secrets in SQLite.

• No direct database access from React.

• No generic SQL or arbitrary filesystem IPC command.

• Every user-owned query is scoped to the active local profile.

• Every file shelf operation preserves original source files unless the user explicitly asks for a separate managed copy.

• Every migration is forward-only, checksummed, tested, and backed up.

• Every cloud feature, if ever introduced, is optional and consent-based.

# Appendix A. Consolidated SQLite DDL

Reference DDL for the core v1 schema. Production migrations should split this into immutable numbered migration files. The catalogue above remains authoritative for semantic rules and extended constraints.

```
PRAGMA foreign_keys = ON;
PRAGMA journal_mode = WAL;
PRAGMA busy_timeout = 5000;

CREATE TABLE app_installation (
  id TEXT PRIMARY KEY NOT NULL,
  created_at TEXT NOT NULL,
  first_seen_version TEXT NOT NULL,
  last_seen_version TEXT NOT NULL,
  install_channel TEXT NOT NULL CHECK (install_channel IN ('stable','beta','dev')),
  platform TEXT NOT NULL DEFAULT 'windows',
  last_migration_at TEXT,
  uninstall_requested_at TEXT
);

CREATE TABLE local_profile (
  id TEXT PRIMARY KEY NOT NULL,
  installation_id TEXT NOT NULL REFERENCES app_installation(id) ON DELETE RESTRICT,
  display_name TEXT NOT NULL DEFAULT 'My MILO',
  windows_sid_hash TEXT,
  auth_mode TEXT NOT NULL DEFAULT 'windows_session'
    CHECK (auth_mode IN ('windows_session')),
  is_default INTEGER NOT NULL DEFAULT 1 CHECK (is_default IN (0,1)),
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  deleted_at TEXT
);
CREATE UNIQUE INDEX ux_local_profile_default
  ON local_profile(installation_id) WHERE is_default = 1;

CREATE TABLE app_session (
  id TEXT PRIMARY KEY NOT NULL,
  local_profile_id TEXT NOT NULL REFERENCES local_profile(id) ON DELETE CASCADE,
  started_at TEXT NOT NULL,
  ended_at TEXT,
  app_version TEXT NOT NULL,
  process_id INTEGER,
  shutdown_reason TEXT,
  last_heartbeat_at TEXT
);
CREATE INDEX ix_app_session_profile_started
  ON app_session(local_profile_id, started_at DESC);

CREATE TABLE auth_event (
  id TEXT PRIMARY KEY NOT NULL,
  local_profile_id TEXT REFERENCES local_profile(id) ON DELETE SET NULL,
  session_id TEXT REFERENCES app_session(id) ON DELETE SET NULL,
  event_type TEXT NOT NULL,
  auth_method TEXT NOT NULL,
  result TEXT NOT NULL CHECK (result IN ('success','denied','error')),
  occurred_at TEXT NOT NULL,
  reason_code TEXT,
  metadata_json TEXT
);
CREATE INDEX ix_auth_event_profile_time
  ON auth_event(local_profile_id, occurred_at DESC);

CREATE TABLE profile_preferences (
  local_profile_id TEXT PRIMARY KEY NOT NULL
    REFERENCES local_profile(id) ON DELETE CASCADE,
  theme_mode TEXT NOT NULL DEFAULT 'dark'
    CHECK (theme_mode IN ('system','dark','light')),
  accent_key TEXT NOT NULL DEFAULT 'aurora',
  launch_at_login INTEGER NOT NULL DEFAULT 0 CHECK (launch_at_login IN (0,1)),
  start_collapsed INTEGER NOT NULL DEFAULT 1 CHECK (start_collapsed IN (0,1)),
  close_behaviour TEXT NOT NULL DEFAULT 'hide_to_tray'
    CHECK (close_behaviour IN ('hide_to_tray','exit')),
  reduce_motion INTEGER NOT NULL DEFAULT 0 CHECK (reduce_motion IN (0,1)),
  sound_enabled INTEGER NOT NULL DEFAULT 1 CHECK (sound_enabled IN (0,1)),
  notification_enabled INTEGER NOT NULL DEFAULT 1 CHECK (notification_enabled IN (0,1)),
  week_start_day INTEGER NOT NULL DEFAULT 1 CHECK (week_start_day BETWEEN 0 AND 6),
  time_format TEXT NOT NULL DEFAULT 'system' CHECK (time_format IN ('system','12h','24h')),
  date_format TEXT NOT NULL DEFAULT 'system',
  timezone_id TEXT,
  updated_at TEXT NOT NULL
);

CREATE TABLE deck_sections (
  id TEXT PRIMARY KEY NOT NULL,
  local_profile_id TEXT NOT NULL REFERENCES local_profile(id) ON DELETE CASCADE,
  section_key TEXT NOT NULL,
```

```
  sort_order INTEGER NOT NULL DEFAULT 0,
  is_visible INTEGER NOT NULL DEFAULT 1 CHECK (is_visible IN (0,1)),
  is_pinned INTEGER NOT NULL DEFAULT 0 CHECK (is_pinned IN (0,1)),
  collapsed_by_default INTEGER NOT NULL DEFAULT 0 CHECK (collapsed_by_default IN (0,1)),
  config_json TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  UNIQUE(local_profile_id, section_key)
);
CREATE INDEX ix_deck_sections_order ON deck_sections(local_profile_id, sort_order);

CREATE TABLE user_shortcuts (
  id TEXT PRIMARY KEY NOT NULL,
  local_profile_id TEXT NOT NULL REFERENCES local_profile(id) ON DELETE CASCADE,
  action_key TEXT NOT NULL,
  accelerator TEXT NOT NULL,
  is_enabled INTEGER NOT NULL DEFAULT 1 CHECK (is_enabled IN (0,1)),
  scope TEXT NOT NULL CHECK (scope IN ('in_app','global')),
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  UNIQUE(local_profile_id, action_key)
);
CREATE UNIQUE INDEX ux_user_shortcut_enabled_accel
  ON user_shortcuts(local_profile_id, accelerator) WHERE is_enabled = 1;

CREATE TABLE tasks (
  id TEXT PRIMARY KEY NOT NULL,
  local_profile_id TEXT NOT NULL REFERENCES local_profile(id) ON DELETE CASCADE,
  title TEXT NOT NULL CHECK (length(title) BETWEEN 1 AND 500),
  description TEXT,
  status TEXT NOT NULL DEFAULT 'open'
    CHECK (status IN ('open','in_progress','completed','archived')),
  priority INTEGER NOT NULL DEFAULT 0 CHECK (priority BETWEEN 0 AND 4),
  due_at TEXT,
  due_timezone_id TEXT,
  completed_at TEXT,
  sort_order INTEGER NOT NULL DEFAULT 0,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  deleted_at TEXT
);
CREATE INDEX ix_tasks_profile_status_order
  ON tasks(local_profile_id, status, sort_order);
CREATE INDEX ix_tasks_due
  ON tasks(local_profile_id, due_at) WHERE deleted_at IS NULL AND due_at IS NOT NULL;

CREATE TABLE task_checklist_items (
  id TEXT PRIMARY KEY NOT NULL,
  task_id TEXT NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
  title TEXT NOT NULL CHECK (length(title) BETWEEN 1 AND 300),
  is_completed INTEGER NOT NULL DEFAULT 0 CHECK (is_completed IN (0,1)),
  sort_order INTEGER NOT NULL DEFAULT 0,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);
CREATE INDEX ix_checklist_task_order ON task_checklist_items(task_id, sort_order);

CREATE TABLE tags (
  id TEXT PRIMARY KEY NOT NULL,
  local_profile_id TEXT NOT NULL REFERENCES local_profile(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  colour_key TEXT,
  created_at TEXT NOT NULL,
  UNIQUE(local_profile_id, name COLLATE NOCASE)
);

CREATE TABLE task_tags (
  task_id TEXT NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
  tag_id TEXT NOT NULL REFERENCES tags(id) ON DELETE CASCADE,
  created_at TEXT NOT NULL,
  PRIMARY KEY(task_id, tag_id)
);

CREATE TABLE reminders (
  id TEXT PRIMARY KEY NOT NULL,
  local_profile_id TEXT NOT NULL REFERENCES local_profile(id) ON DELETE CASCADE,
  title TEXT NOT NULL CHECK (length(title) BETWEEN 1 AND 300),
  message TEXT,
  schedule_type TEXT NOT NULL
    CHECK (schedule_type IN ('once','daily','weekly','monthly','custom')),
  due_at TEXT,
  local_time TEXT,
  timezone_id TEXT,
  recurrence_json TEXT,
  is_enabled INTEGER NOT NULL DEFAULT 1 CHECK (is_enabled IN (0,1)),
  sound_key TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  deleted_at TEXT
```

```
);
CREATE INDEX ix_reminders_enabled
  ON reminders(local_profile_id, is_enabled);

CREATE TABLE reminder_occurrences (
  id TEXT PRIMARY KEY NOT NULL,
  reminder_id TEXT NOT NULL REFERENCES reminders(id) ON DELETE CASCADE,
  scheduled_for TEXT NOT NULL,
  status TEXT NOT NULL
    CHECK (status IN ('scheduled','firing','delivered','snoozed','dismissed','missed','cancelled')),
  delivered_at TEXT,
  snoozed_until TEXT,
  dismissed_at TEXT,
  attempt_count INTEGER NOT NULL DEFAULT 0 CHECK (attempt_count >= 0),
  last_error_code TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  UNIQUE(reminder_id, scheduled_for)
);
CREATE INDEX ix_reminder_occurrences_due
  ON reminder_occurrences(status, scheduled_for);

CREATE TABLE timers (
  id TEXT PRIMARY KEY NOT NULL,
  local_profile_id TEXT NOT NULL REFERENCES local_profile(id) ON DELETE CASCADE,
  label TEXT,
  duration_ms INTEGER NOT NULL CHECK (duration_ms BETWEEN 1 AND 604800000),
  repeat_enabled INTEGER NOT NULL DEFAULT 0 CHECK (repeat_enabled IN (0,1)),
  sound_key TEXT,
  sort_order INTEGER NOT NULL DEFAULT 0,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

CREATE TABLE timer_runs (
  id TEXT PRIMARY KEY NOT NULL,
  timer_id TEXT NOT NULL REFERENCES timers(id) ON DELETE CASCADE,
  status TEXT NOT NULL CHECK (status IN ('running','paused','completed','cancelled')),
  started_at TEXT NOT NULL,
  deadline_at TEXT,
  remaining_ms INTEGER NOT NULL CHECK (remaining_ms >= 0),
  paused_at TEXT,
  completed_at TEXT,
  last_notified_at TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);
CREATE INDEX ix_timer_runs_timer_created ON timer_runs(timer_id, created_at DESC);
CREATE INDEX ix_timer_runs_running_deadline
  ON timer_runs(status, deadline_at) WHERE status = 'running';

CREATE TABLE alarms (
  id TEXT PRIMARY KEY NOT NULL,
  local_profile_id TEXT NOT NULL REFERENCES local_profile(id) ON DELETE CASCADE,
  label TEXT,
  local_time TEXT NOT NULL,
  timezone_id TEXT,
  repeat_mask INTEGER NOT NULL CHECK (repeat_mask BETWEEN 0 AND 127),
  one_time_date TEXT,
  is_enabled INTEGER NOT NULL DEFAULT 1 CHECK (is_enabled IN (0,1)),
  snooze_minutes INTEGER NOT NULL DEFAULT 5 CHECK (snooze_minutes BETWEEN 1 AND 60),
  sound_key TEXT,
  next_fire_at TEXT,
  last_fired_at TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);
CREATE INDEX ix_alarms_next_fire
  ON alarms(local_profile_id, is_enabled, next_fire_at);

CREATE TABLE staged_files (
  id TEXT PRIMARY KEY NOT NULL,
  local_profile_id TEXT NOT NULL REFERENCES local_profile(id) ON DELETE CASCADE,
  source_path TEXT NOT NULL,
  display_name TEXT NOT NULL,
  extension TEXT,
  mime_type TEXT,
  size_bytes INTEGER CHECK (size_bytes IS NULL OR size_bytes >= 0),
  modified_at_source TEXT,
  content_hash TEXT,
  status TEXT NOT NULL
    CHECK (status IN ('staged','missing','unavailable','removed_from_shelf','expired')),
  staged_at TEXT NOT NULL,
  last_verified_at TEXT,
  expires_at TEXT,
  removed_at TEXT
);
CREATE INDEX ix_staged_files_profile_status
  ON staged_files(local_profile_id, status, staged_at DESC);
```

```
CREATE TABLE staged_file_events (
  id TEXT PRIMARY KEY NOT NULL,
  staged_file_id TEXT REFERENCES staged_files(id) ON DELETE SET NULL,
  local_profile_id TEXT NOT NULL REFERENCES local_profile(id) ON DELETE CASCADE,
  event_type TEXT NOT NULL,
  occurred_at TEXT NOT NULL,
  result TEXT NOT NULL CHECK (result IN ('success','cancelled','failed')),
  error_code TEXT,
  details_json TEXT
);
CREATE INDEX ix_staged_file_events_profile_time
  ON staged_file_events(local_profile_id, occurred_at DESC);

CREATE TABLE media_favorites (
  id TEXT PRIMARY KEY NOT NULL,
  local_profile_id TEXT NOT NULL REFERENCES local_profile(id) ON DELETE CASCADE,
  provider_key TEXT NOT NULL,
  provider_item_id TEXT,
  title TEXT NOT NULL,
  artist TEXT,
  album TEXT,
  artwork_cache_key TEXT,
  created_at TEXT NOT NULL
);
CREATE UNIQUE INDEX ux_media_favorite_provider_item
  ON media_favorites(local_profile_id, provider_key, provider_item_id)
  WHERE provider_item_id IS NOT NULL;

CREATE TABLE avatar_settings (
  local_profile_id TEXT PRIMARY KEY NOT NULL
    REFERENCES local_profile(id) ON DELETE CASCADE,
  avatar_key TEXT NOT NULL,
  enabled INTEGER NOT NULL DEFAULT 1 CHECK (enabled IN (0,1)),
  animation_level TEXT NOT NULL DEFAULT 'full'
    CHECK (animation_level IN ('full','reduced','minimal','off')),
  idle_behavior TEXT NOT NULL DEFAULT 'subtle'
    CHECK (idle_behavior IN ('subtle','still')),
  sound_reactions_enabled INTEGER NOT NULL DEFAULT 0 CHECK (sound_reactions_enabled IN (0,1)),
  last_selected_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

CREATE TABLE notifications (
  id TEXT PRIMARY KEY NOT NULL,
  local_profile_id TEXT NOT NULL REFERENCES local_profile(id) ON DELETE CASCADE,
  source_type TEXT NOT NULL,
  source_id TEXT,
  title TEXT NOT NULL,
  body TEXT,
  status TEXT NOT NULL CHECK (status IN ('pending','shown','dismissed','actioned','failed')),
  scheduled_at TEXT,
  shown_at TEXT,
  dismissed_at TEXT,
  action_key TEXT,
  created_at TEXT NOT NULL
);
CREATE INDEX ix_notifications_queue
  ON notifications(local_profile_id, status, scheduled_at);

CREATE TABLE quick_actions (
  id TEXT PRIMARY KEY NOT NULL,
  local_profile_id TEXT NOT NULL REFERENCES local_profile(id) ON DELETE CASCADE,
  action_key TEXT NOT NULL,
  label_override TEXT,
  sort_order INTEGER NOT NULL DEFAULT 0,
  is_enabled INTEGER NOT NULL DEFAULT 1 CHECK (is_enabled IN (0,1)),
  config_json TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  UNIQUE(local_profile_id, action_key)
);

CREATE TABLE permission_grants (
  id TEXT PRIMARY KEY NOT NULL,
  local_profile_id TEXT NOT NULL REFERENCES local_profile(id) ON DELETE CASCADE,
  permission_key TEXT NOT NULL,
  decision TEXT NOT NULL CHECK (decision IN ('granted','denied','revoked')),
  scope_json TEXT,
  decided_at TEXT NOT NULL,
  expires_at TEXT,
  revoked_at TEXT,
  source TEXT NOT NULL
);
CREATE INDEX ix_permission_grants_key_time
  ON permission_grants(local_profile_id, permission_key, decided_at DESC);

CREATE TABLE export_jobs (
  id TEXT PRIMARY KEY NOT NULL,
  local_profile_id TEXT NOT NULL REFERENCES local_profile(id) ON DELETE CASCADE,
```

```
  format TEXT NOT NULL CHECK (format IN ('json','csv','zip')),
  status TEXT NOT NULL CHECK (status IN ('queued','running','completed','failed','cancelled')),
  destination_path TEXT,
  requested_at TEXT NOT NULL,
  completed_at TEXT,
  expires_at TEXT,
  error_code TEXT,
  includes_file_paths INTEGER NOT NULL DEFAULT 0 CHECK (includes_file_paths IN (0,1))
);
CREATE INDEX ix_export_jobs_profile_time
  ON export_jobs(local_profile_id, requested_at DESC);

CREATE TABLE app_events (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  session_id TEXT REFERENCES app_session(id) ON DELETE SET NULL,
  local_profile_id TEXT REFERENCES local_profile(id) ON DELETE SET NULL,
  event_name TEXT NOT NULL,
  severity TEXT NOT NULL CHECK (severity IN ('debug','info','warning','error')),
  occurred_at TEXT NOT NULL,
  error_code TEXT,
  component TEXT,
  duration_ms INTEGER,
  details_json TEXT
);
CREATE INDEX ix_app_events_time ON app_events(occurred_at DESC);
CREATE INDEX ix_app_events_severity_time ON app_events(severity, occurred_at DESC);

CREATE TABLE schema_migrations (
  version INTEGER PRIMARY KEY NOT NULL,
  name TEXT NOT NULL UNIQUE,
  checksum TEXT NOT NULL,
  applied_at TEXT NOT NULL,
  execution_ms INTEGER,
  app_version TEXT NOT NULL
);
```

# Appendix B. Enum and state definitions

| Field | Allowed values / encoding |
|---|---|
| install_channel | stable \| beta \| dev |
| auth_mode | windows_session |
| auth_event.event_type | startup_accepted \| profile_opened \| reauth_requested \| reauth_succeeded \| reauth_failed \| profile_locked \| profile_deleted |
| auth_event.auth_method | windows_session \| windows_hello \| os_credential_prompt \| none |
| auth_event.result | success \| denied \| error |
| theme_mode | system \| dark \| light |
| close_behaviour | hide_to_tray \| exit |
| task.status | open \| in_progress \| completed \| archived |
| task.priority | 0 none \| 1 low \| 2 normal \| 3 high \| 4 urgent |
| reminder.schedule_type | once \| daily \| weekly \| monthly \| custom |
| reminder_occurrence.status | scheduled \| firing \| delivered \| snoozed \| dismissed \| missed \| cancelled |
| timer_run.status | running \| paused \| completed \| cancelled |
| staged_file.status | staged \| missing \| unavailable \| removed_from_shelf \| expired |
| staged_file_event.result | success \| cancelled \| failed |
| notification.status | pending \| shown \| dismissed \| actioned \| failed |
| permission_grant.decision | granted \| denied \| revoked |
| export_job.status | queued \| running \| completed \| failed \| cancelled |
| severity | debug \| info \| warning \| error |

## B.1 Weekday repeat mask

`alarms.repeat_mask` is a seven-bit integer. Define bit 0 = Monday, bit 1 = Tuesday, bit 2 = Wednesday, bit 3 = Thursday, bit 4 = Friday, bit 5 = Saturday, bit 6 = Sunday. A value of 0 means a one-time alarm and requires `one_time_date`. Example: weekdays Monday–Friday = 31.

## B.2 Timestamp rules

• Persist UTC instants as RFC 3339 strings ending in `Z`.

• Persist user-entered recurring wall-clock time separately from the timezone identifier.

• Do not store local timestamps without their time-zone context.

• DST gaps and overlaps must have deterministic product rules and tests.

• Use monotonic clocks only for in-process elapsed-time measurement; persist wall-clock deadlines for recovery.

# Appendix C. Reference documentation

The following authoritative references inform the implementation choices. They should be rechecked against the exact dependency versions selected during development.

**Tauri v2 Security** — Trust boundaries, IPC, CSP and security concepts.
https://v2.tauri.app/security/

**Tauri v2 Capabilities** — Window/webview-specific permission grants and capability configuration.
https://tauri.app/security/capabilities/

**Tauri v2 Permissions** — Command permission descriptions and scopes.
https://tauri.app/security/permissions/

**SQLite File Format / WAL** — Write-ahead logging and database file behaviour.
https://www.sqlite.org/fileformat.html

**SQLite Foreign Key Support** — Foreign key constraints and enforcement.
https://www.sqlite.org/foreignkeys.html

**Microsoft CryptProtectData** — Windows Data Protection API for user- or machine-scoped secret protection.
https://learn.microsoft.com/en-us/windows/win32/api/dpapi/nf-dpapi-cryptprotectdata

**Microsoft Credential Manager** — Windows generic credentials and credential management concepts.
https://learn.microsoft.com/en-us/windows/win32/secauthn/kinds-of-credentials

## Implementation caveat

This document is an engineering specification and reference architecture, not a claim that MILO has already implemented or independently security-audited these controls. The implementation team must validate current API behaviour, dependency versions, Windows support matrices, and threat assumptions before release.

Final architecture rule: keep local data local, keep secrets out of the database, keep the frontend away from privileged storage, and make every cross-boundary action explicit, narrow, and testable.